



K-Nearest Neighbors (KNN) – Customer Churn Prediction

Practical Implementation using Churn_Modelling.csv

Problem Statement

A bank wants to predict whether a customer will **churn (exit)** based on customer information such as credit score, geography, gender, age, tenure, balance, number of products, credit card ownership, activity status, and estimated salary.

We will build a **K-Nearest Neighbors (KNN) Classification model** and evaluate it using standard classification metrics. Finally, we will tune the KNN hyperparameters using **GridSearchCV** and **RandomizedSearchCV**.

Practical Workflow

Data Collection → Data Preprocessing → Model Building → Testing → Evaluation → Hyperparameter Tuning

Learning Objectives

By the end of this practical, students should be able to:

- Understand the working principle of KNN.
- Preprocess a real-world churn dataset.
- Handle missing values and categorical variables.
- Detect potential outliers using visualization.
- Understand why feature scaling is important for KNN.
- Build and train a KNN classification model.
- Evaluate the model using accuracy, confusion matrix, precision, recall and F1-score.
- Understand the significance of **K**.
- Use **GridSearchCV** and **RandomizedSearchCV** for hyperparameter tuning.

Data Collection

The dataset used in this practical is `Churn_Modelling.csv`.

Target Variable

`Exited`

- `0` → Customer did not churn
- `1` → Customer churned

Important Features

Feature	Description
CreditScore	Customer's credit score
Geography	Customer's country
Gender	Customer's gender
Age	Customer's age
Tenure	Number of years with the bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Whether customer has a credit card
IsActiveMember	Whether customer is active
EstimatedSalary	Estimated salary

Note: Keep `Churn_Modelling.csv` in the same folder as this notebook.

In [1]: *# 1.1 Import Libraries and Dataset*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
```

```

        confusion_matrix,
        classification_report,
        precision_score,
        recall_score,
        f1_score
    )

# Load dataset
df = pd.read_csv("Churn_Modelling.csv")

print("Dataset Shape:", df.shape)
df.head()

```

Dataset Shape: (10000, 14)

Out[1]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	T
0	1	15634602	Hargrave	619	France	Female	42.0	
1	2	15647311	Hill	608	Spain	Female	41.0	
2	3	15619304	Onio	502	France	Female	42.0	
3	4	15701354	Boni	699	France	Female	39.0	
4	5	15737888	Mitchell	850	Spain	Female	43.0	

1.2 Checking for Null Values

Before building the model, check whether any columns contain missing values.

For this dataset, missing values may occur in `Age` and `Gender`.

- Numerical feature → fill with **median**
- Categorical feature → fill with **mode**

```

In [4]: # Check missing values

print(df.isnull().sum())

```

```
RowNumber      0
CustomerId     0
Surname        0
CreditScore    0
Geography      0
Gender         3
Age           3
Tenure        0
Balance       0
NumOfProducts 0
HasCrCard     0
IsActiveMember 0
EstimatedSalary 0
Exited        0
dtype: int64
```

```
In [6]: # Handle missing values
```

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

print("Missing values after treatment:")
print(df.isnull().sum())
```

```
Missing values after treatment:
```

```
RowNumber      0
CustomerId     0
Surname        0
CreditScore    0
Geography      0
Gender         0
Age           0
Tenure        0
Balance       0
NumOfProducts 0
HasCrCard     0
IsActiveMember 0
EstimatedSalary 0
Exited        0
dtype: int64
```

1.3 Data Visualization

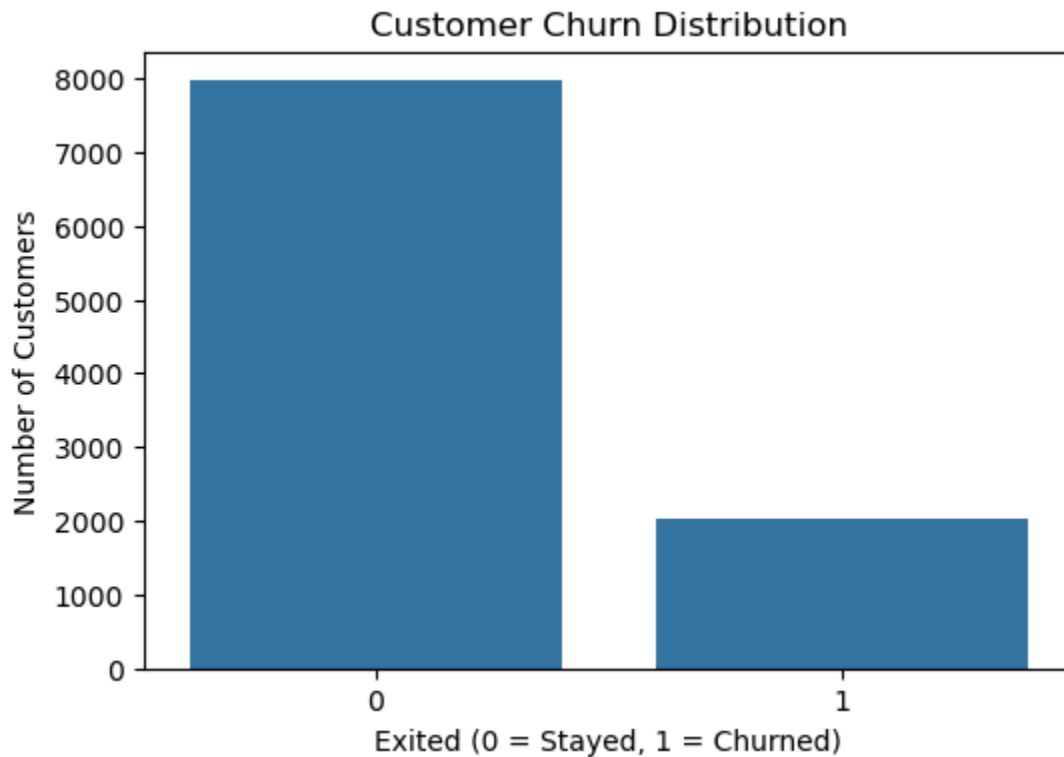
Visualization helps us understand:

- Class distribution
- Relationships between important features and churn
- Possible differences between churned and non-churned customers

```
In [9]: # Churn distribution
```

```
plt.figure(figsize=(6, 4))
sns.countplot(x='Exited', data=df)
plt.title("Customer Churn Distribution")
plt.xlabel("Exited (0 = Stayed, 1 = Churned)")
plt.ylabel("Number of Customers")
plt.show()

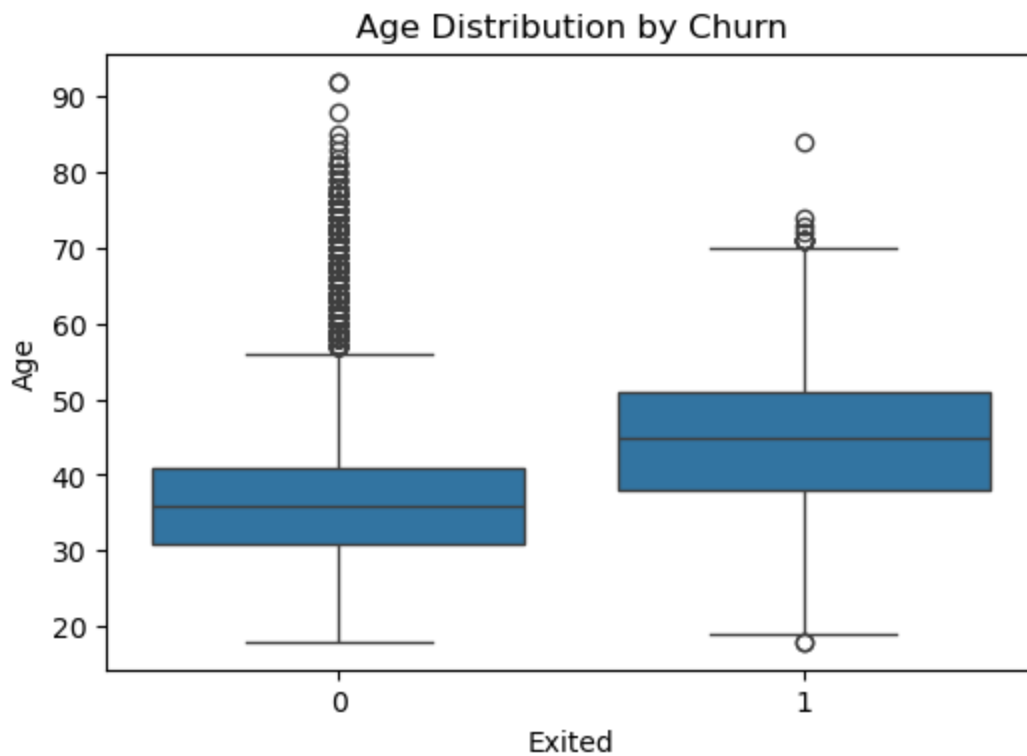
print(df['Exited'].value_counts())
```



```
Exited
0    7963
1     2037
Name: count, dtype: int64
```

In [11]: *# Age distribution by churn*

```
plt.figure(figsize=(6, 4))
sns.boxplot(x='Exited', y='Age', data=df)
plt.title("Age Distribution by Churn")
plt.xlabel("Exited")
plt.ylabel("Age")
plt.show()
```



1.4 Outlier Detection

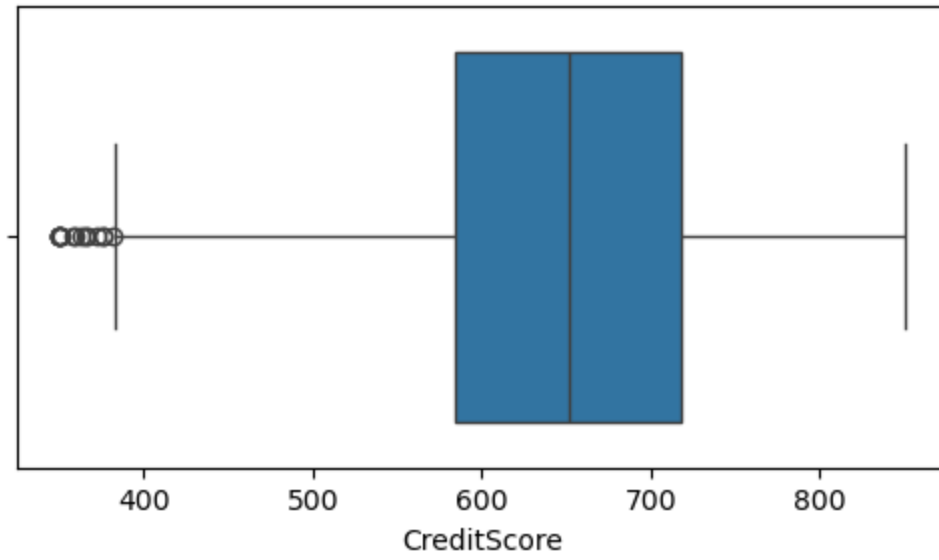
KNN is a **distance-based algorithm**, so extreme observations can influence distance calculations.

We use boxplots to identify potential outliers. However, we should **not automatically remove every outlier** because an extreme value may be a genuine customer observation.

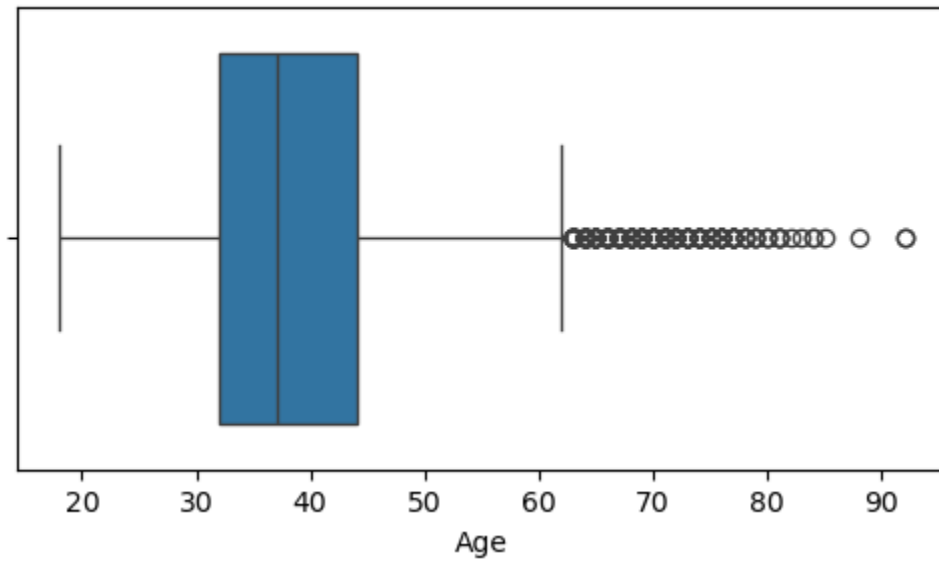
In [17]: *# Boxplots for important numerical variables*

```
numeric_columns = [  
    'CreditScore',  
    'Age',  
    'Tenure',  
    'Balance',  
    'NumOfProducts',  
    'EstimatedSalary'  
]  
  
for column in numeric_columns:  
    plt.figure(figsize=(6, 3))  
    sns.boxplot(x=df[column])  
    plt.title(f"Boxplot of {column}")  
    plt.show()
```

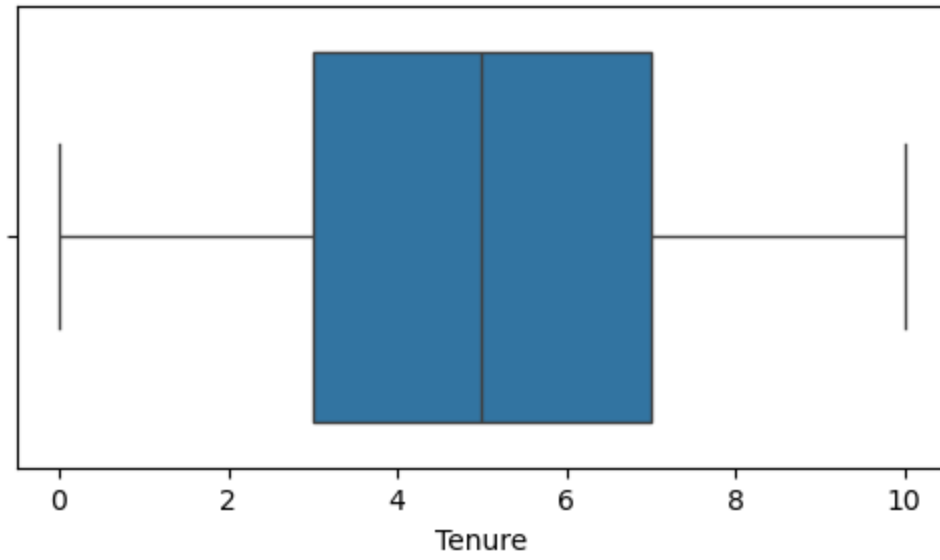
Boxplot of CreditScore



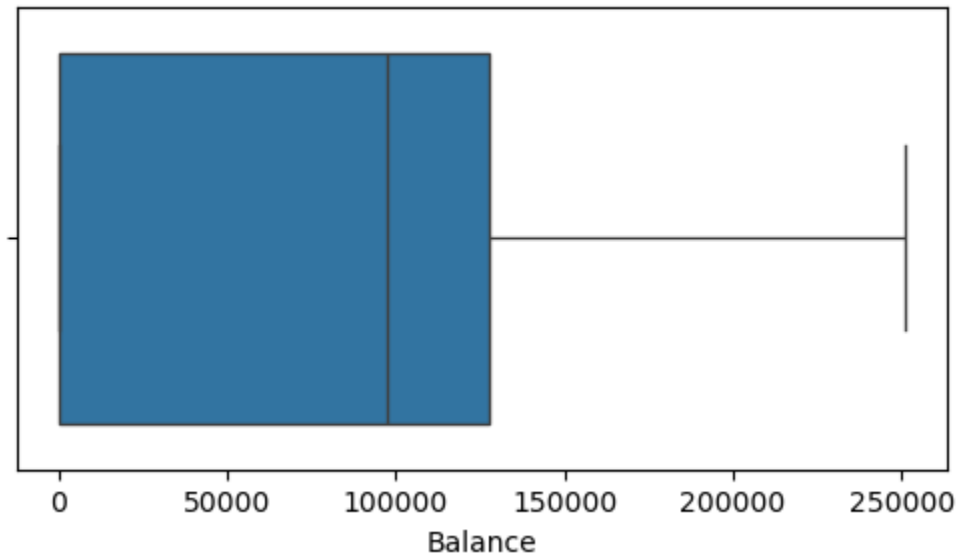
Boxplot of Age

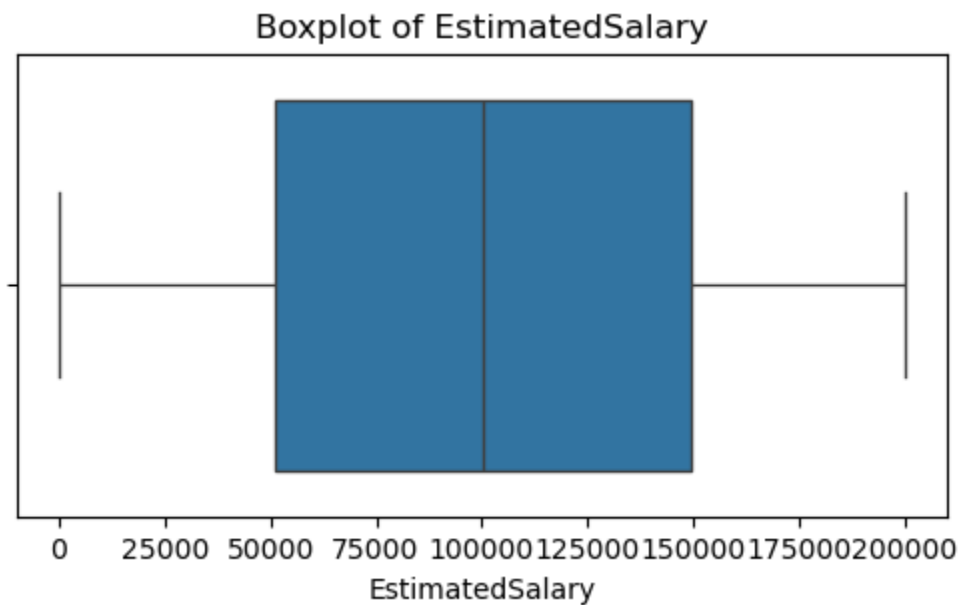
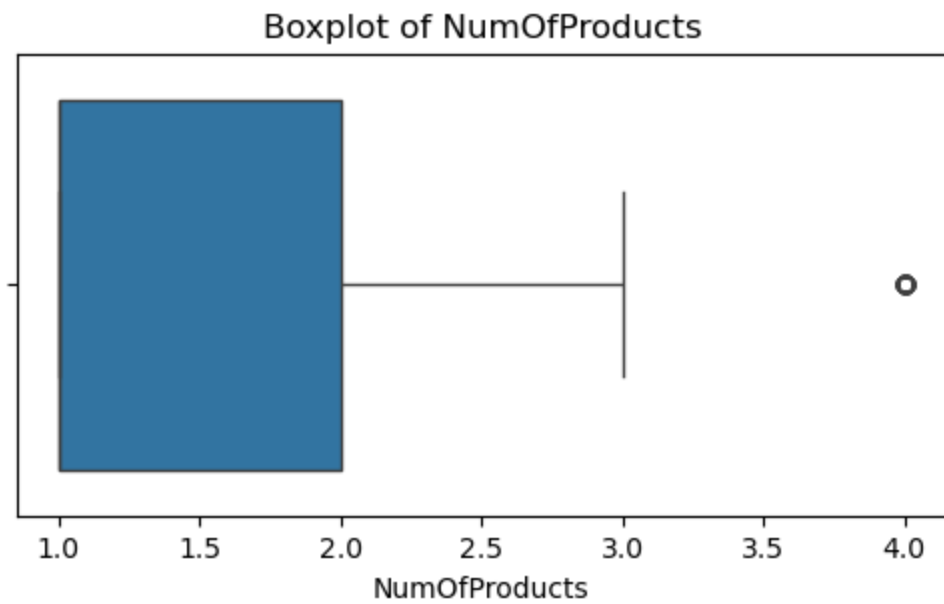


Boxplot of Tenure



Boxplot of Balance





1.5 Splitting Independent and Dependent Variables

The target variable is `Exited`.

We should also remove identifier columns such as:

- `RowNumber`
- `CustomerId`
- `Surname`

These columns identify customers but do not provide useful predictive information

for this model.

```
In [20]: # Independent variables
X = df.drop(
    ['RowNumber', 'CustomerId', 'Surname', 'Exited'],
    axis=1
)

# Dependent variable
y = df['Exited']

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (10000, 10)
y shape: (10000,)
```

1.6 Encoding Categorical Variables

The columns `Geography` and `Gender` contain categorical values.

KNN requires numerical input, so we use **One-Hot Encoding**.

`drop_first=True` removes one category from each encoded group to avoid unnecessary redundancy.

```
In [23]: # One-hot encode categorical variables

X = pd.get_dummies(
    X,
    columns=['Geography', 'Gender'],
    drop_first=True
)

print("Encoded feature columns:")
print(X.columns.tolist())

X.head()
```

```
Encoded feature columns:
['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary', 'Geography_Germany', 'Geography_Spain', 'Gender_Male']
```

Out[23]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMe
0	619	42.0	2	0.00	1	1	
1	608	41.0	1	83807.86	1	0	
2	502	42.0	8	159660.80	3	1	
3	699	39.0	1	0.00	2	0	
4	850	43.0	2	125510.82	1	1	

1.7 Splitting Data into Training and Testing Sets

We use:

- **80% Training data**
- **20% Testing data**

`stratify=y` helps preserve the proportion of churned and non-churned customers in both sets.

```
In [26]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)
```

Training data: (8000, 11)

Testing data : (2000, 11)

1.8 Feature Scaling

Why is scaling important for KNN?

KNN calculates distances between data points.

For example:

- Age → approximately 18-90
- Balance → can be in hundreds of thousands
- EstimatedSalary → can be in hundreds of thousands

Without scaling, large-valued features can dominate the distance calculation.

Therefore, we use **StandardScaler**.

Important rule

Fit the scaler only on training data:

```
fit_transform(X_train)
```

Then use the same fitted scaler to transform test data:

```
transform(X_test)
```

```
In [29]: # Feature scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training data scaled successfully.")
print("Testing data scaled successfully.")
```

Training data scaled successfully.
Testing data scaled successfully.

2. Model Building

Import and Initialize KNN

We initially choose:

K = 5

K represents the number of nearest neighbors used for prediction.

```
In [32]: # Initialize KNN classifier

knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train_scaled, y_train)

print("KNN model trained successfully.")
```

KNN model trained successfully.

3. Testing the Model

Use the trained KNN model to predict churn for the unseen test data.

```
In [35]: # Make predictions

y_pred = knn.predict(X_test_scaled)

print("First 20 predictions:")
print(y_pred[:20])

print("\nFirst 20 actual values:")
print(y_test.values[:20])
```

```
First 20 predictions:
[0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0]
```

```
First 20 actual values:
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0]
```

4. Evaluation of the Model

We evaluate the model using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

For a churn problem, **recall for the churn class (Exited = 1) can be especially important**, because failing to identify a customer who will churn may result in a missed retention opportunity.

```
In [38]: # Accuracy

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.824
```

```
In [40]: # Confusion Matrix

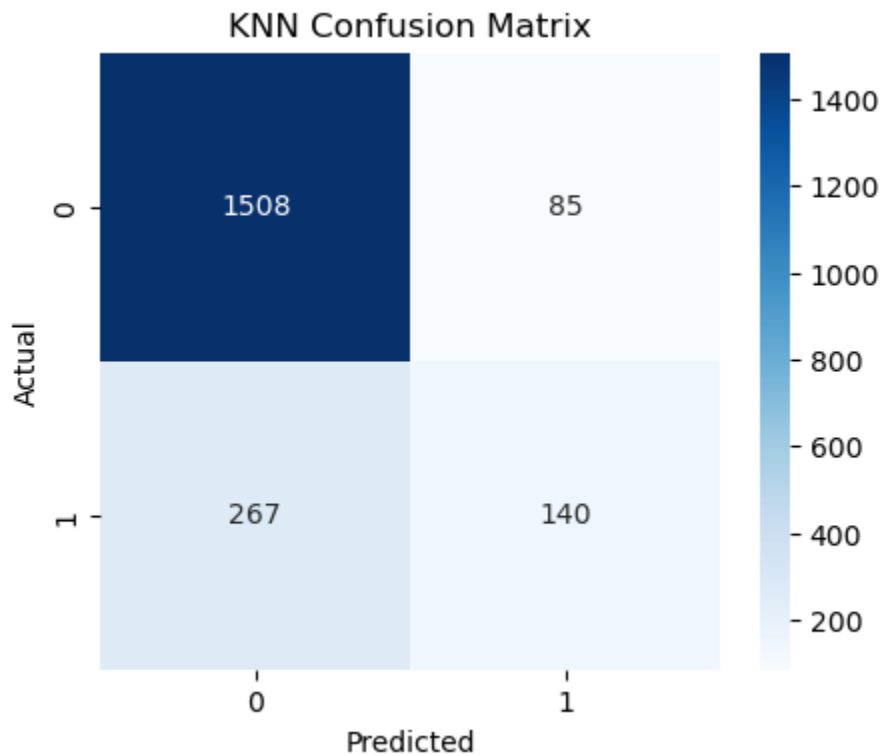
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
plt.figure(figsize=(5, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
```

```
[[1508  85]
 [ 267 140]]
```



```
In [42]: # Classification Report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.85	0.95	0.90	1593
1	0.62	0.34	0.44	407
accuracy			0.82	2000
macro avg	0.74	0.65	0.67	2000
weighted avg	0.80	0.82	0.80	2000

```
In [44]: # Individual evaluation metrics
```

```
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

Accuracy : 0.824
Precision: 0.6222222222222222
Recall : 0.343980343980344
F1 Score : 0.4430379746835443

Understanding the Significance of K

The value of **K** strongly affects KNN.

Small K

A very small K, such as **K=1**, makes the model highly sensitive to individual training points and noise. This can cause **overfitting**.

Large K

A very large K considers many training points. The model becomes smoother but may ignore important local patterns, causing **underfitting**.

Therefore

Small K → Low Bias, High Variance → Possible Overfitting

Large K → High Bias, Low Variance → Possible Underfitting

The best K should be selected using validation or cross-validation rather than guessing.

5. Hyperparameter Tuning

Instead of manually selecting K, we can use automated search methods.

Two important techniques are:

GridSearchCV

Tests **every combination** of the specified hyperparameter values.

RandomizedSearchCV

Tests a specified number of **randomly selected combinations** from the parameter distributions.

Both methods use cross-validation to compare model configurations.

GridSearchCV

Suppose we want to test:

- `n_neighbors`
- `weights`
- `metric`

GridSearchCV tries every combination we specify.

For example, if we have:

- 7 K values
- 2 weight options
- 2 distance metrics

then:

$7 \times 2 \times 2 = 28$ combinations

With `cv=5`, each combination is evaluated using 5-fold cross-validation.

```
In [51]: from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train_scaled, y_train)
```



```

print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Score:")
print(grid_search.best_score_)

```

Best Parameters:
{'metric': 'euclidean', 'n_neighbors': 15, 'weights': 'distance'}

Best Cross-Validation Score:
0.8335000000000001

In [52]: *# Evaluate the best GridSearchCV model on test data*

```

best_knn_grid = grid_search.best_estimator_

y_pred_grid = best_knn_grid.predict(X_test_scaled)

print("Test Accuracy:",
      accuracy_score(y_test, y_pred_grid))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_grid))

```

Test Accuracy: 0.8375

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.97	0.91	1593
1	0.75	0.30	0.43	407
accuracy			0.84	2000
macro avg	0.80	0.64	0.67	2000
weighted avg	0.83	0.84	0.81	2000

What is 5-Fold Cross-Validation?

With `cv=5`, the training data is divided into 5 parts.

Each part is used once as validation data while the remaining 4 parts are used for training.

The five validation scores are then averaged.

Cross-validation gives a more reliable estimate of model performance than relying on a single train-validation split.

RandomizedSearchCV

GridSearchCV can become expensive when the number of possible combinations is large.

RandomizedSearchCV solves this by testing only a specified number of randomly selected combinations.

For example:

```
n_iter=20
```

means that 20 parameter combinations are sampled and evaluated.

```
In [57]: from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distributions = {
    'n_neighbors': randint(1, 50),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

random_search = RandomizedSearchCV(
    estimator=KNeighborsClassifier(),
    param_distributions=param_distributions,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train_scaled, y_train)

print("Best Parameters:")
print(random_search.best_params_)

print("\nBest Cross-Validation Score:")
print(random_search.best_score_)
```

```
Best Parameters:
{'metric': 'euclidean', 'n_neighbors': 14, 'weights': 'distance'}
```

```
Best Cross-Validation Score:
0.83375
```

```
In [58]: # Evaluate the best RandomizedSearchCV model

best_knn_random = random_search.best_estimator_
```

```

y_pred_random = best_knn_random.predict(X_test_scaled)

print("Test Accuracy:",
      accuracy_score(y_test, y_pred_random))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_random))

```

Test Accuracy: 0.837

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.97	0.90	1593
1	0.73	0.31	0.44	407
accuracy			0.84	2000
macro avg	0.79	0.64	0.67	2000
weighted avg	0.82	0.84	0.81	2000

GridSearchCV vs RandomizedSearchCV

Feature	GridSearchCV	RandomizedSearchCV
Search	Every specified combination	Random combinations
Computation	Can be high	Usually lower
Search control	<code>param_grid</code>	<code>param_distributions</code>
Number tested	All combinations	Controlled by <code>n_iter</code>
Best for	Smaller search spaces	Larger search spaces

Easy way to remember

GridSearchCV: "Try everything I specify."

RandomizedSearchCV: "Try a selected number of random combinations."

Final Practical Workflow

Churn_Modelling.csv

↓

Data Collection

↓

Data Preprocessing

↓

```
Null Value Treatment
↓
Data Visualization
↓
Outlier Detection
↓
Separate X and y
↓
Remove Identifier Columns
↓
Encoding
↓
Train-Test Split
↓
Feature Scaling
↓
KNN Model Building
↓
Testing
↓
Evaluation
↓
Hyperparameter Tuning
↓
GridSearchCV / RandomizedSearchCV
↓
Best Parameters
↓
Final KNN Model
```

Key Takeaways

1. **KNN is a supervised, distance-based algorithm.**
2. **K is the number of neighbors used for prediction.**
3. **Feature scaling is very important for KNN.**
4. **Small K may cause overfitting.**
5. **Large K may cause underfitting.**
6. **Classification uses majority voting.**
7. **GridSearchCV tests all specified combinations.**
8. **RandomizedSearchCV tests a selected number of random combinations.**
9. **Cross-validation helps select hyperparameters more reliably.**
10. **For churn prediction, do not rely only on accuracy; examine precision, recall, F1-score and the confusion matrix.**

In []: